

Project Proposal
BSc (Hons) in Artificial Intelligence
Prompt Injection Detection Using Small Language Models

Fernando W.W.A.H.H.
215523H
Department of Computational Mathematics
Faculty of Information Technology
University of Moratuwa
2026

Table of Contents

1. Introduction.....	1
2. Objectives.....	1
3. Prompt Injection Detection: Current Practices and Challenges.....	1
3.1. Background and Existing Research.....	1
3.2. Problem Summary, Research Gap and Selected Technology.....	2
4. Research Gap.....	2
5. Proposed Research Outcome.....	3
5.1. Overview.....	3
5.2. Methodology and Architecture.....	3
5.3. Process Flow and Components Overview.....	4
6. Resource Requirements.....	4
7. Summary.....	4
8. References.....	4
9. Appendix.....	6

1. Introduction

Large Language Models (LLMs) are widely used in real-world applications such as chatbots, RAG systems, and agentic applications. Despite their power, LLMs introduce new security vulnerabilities and threats, most notably Prompt Injection. This inference-time attack uses crafted malicious prompts to manipulate the model into producing outputs that violate its intended behavior. Recent research shows that prompt injection can bypass standard defenses, especially in systems that expose prompts, tools, or sensitive data. With the growing use of LLMs in enterprise and safety-critical environments, this attack presents a serious security risk.

Existing detection methods range from rule-based and machine learning approaches to LLM-based detectors, but they face challenges such as high false positives, limited adaptability, and high computational cost. As many rely on large or training-heavy models, they are often unsuitable for real-time deployment. This project proposes a prompt injection detection framework based on Small Language Models (SLMs), designed to function as a lightweight guardrail for LLM-based applications. By leveraging fine-tuned SLMs, the framework aims to deliver efficient, deployable, and adaptable prompt injection detection for real-world systems.

2. Objectives

The primary objectives of this project are:

- To conduct a critical review of existing prompt injection detection and prevention techniques.
- To examine Small Language Models (SLMs) and assess their suitability as security guardrails.
- To design and implement a sequential specialist SLM ensemble for real-time prompt injection detection, using Parameter-Efficient Fine-Tuning (PEFT) techniques
- To evaluate the proposed system using standard security metrics, including precision, recall, F1-score, and false positive rate.
- To assess the practical deployability of the solution as a real-world detection mechanism or Python library.

3. Prompt Injection Detection: Current Practices and Challenges

3.1. Background and Existing Research

The proliferation of agentic AI systems has necessitated robust defensive frameworks to mitigate prompt injection—the primary security vulnerability in Large Language Model (LLM) integrations [8]. As observed by Sharma and Mehta [4], the focus of the industry is shifting from general-purpose LLM orchestration toward specialized Small Language Models (SLMs) capable of handling repetitive subtasks with higher efficiency and lower latency. Within this domain, detection strategies have evolved from simple keyword filtering to complex architectural interventions. DataSentinel [17], a game-theoretic approach that utilizes minimax optimization to fine-tune models to be intentionally "vulnerable" to catch adaptive attacks. Simultaneously, internal state analysis has emerged as a viable training-free alternative; Hung et al. [15] introduced the Attention Tracker, which identifies a "distraction

effect" where specific attention heads shift focus from original instructions to injected payloads. Furthermore, structural approaches such as the Signed-Prompt method proposed by Suo [7] attempt to verify instruction sources by replacing sensitive commands with unique, non-natural character combinations to prevent the execution of unauthorized third-party inputs.

While detection scales effectively, trade-offs remain: game-theoretic models offer adaptive resilience at high computational costs, whereas efficient, training-free attention-based methods require internal access incompatible with proprietary APIs. Traditional machine learning classifiers, including LSTM and Random Forest models explored by Shaheer et al. [12], provide baseline protection but often fail to generalize to the nuanced, context-aligned attacks typical of multi-turn conversational agents. While the PromptShield framework [8] successfully reduces False Positive Rates (FPR) in structured application data, it remains susceptible to sophisticated "jailbreak-style" bypasses.

3.2. Problem Summary, Research Gap and Selected Technology

A critical gap remains for lightweight, deployable frameworks that offer interpretability. Existing high-performance models are either too computationally expensive for edge use or provide insufficient context for human triage due to binary output limitations. [9]. Furthermore, there is a lack of localized security middleware that can operate in black-box environments without requiring access to the internal logits or attention maps of the primary foundation model. Consequently, there is an urgent need for a framework that leverages quantized SLMs to achieve frontier-level accuracy with a specific emphasis on low FPR and generative reasoning for production-grade agentic systems.

This project selects Small Language Models (SLMs) with parameter counts lower than or equal to 1B as the core technological foundation. The rationale for this selection is grounded in recent compute-optimal scaling findings [10], [1], which show that well-trained SLMs can achieve reasoning performance comparable to, or exceeding, models an order of magnitude larger. By leveraging Parameter-Efficient Fine-Tuning (PEFT) techniques such as LoRA and QLoRA [3], these compact models can be effectively specialized into high-speed security monitors with minimal training overhead. Their low-latency inference characteristics make them suitable for real-time prompt filtering, even in resource-constrained environments. Moreover, the generative capabilities of modern SLMs enable the production of lightweight, supporting human-in-the-loop security analysis while avoiding the high computational and token costs typically associated with large-scale LLMs [5].

4. Research Gap

Prompt injection attacks exploit the natural language interfaces of LLM-based applications to override system instructions, bypass safety constraints, or manipulate model behavior in unintended ways. Although several detection and prevention techniques have been proposed, many existing approaches are computationally expensive, overly rigid, or, limiting their effectiveness against evolving attack patterns. Consequently, there is a growing need for an efficient, adaptable, and easily deployable detection system that can identify prompt injection

attacks in real time while maintaining low latency and minimal impact on overall system performance.

5. Proposed Research Outcome

5.1. Overview

This project proposes a real-time prompt injection detection framework based on Small Language Models (SLMs), designed to operate as a lightweight and deployable security layer for LLM-integrated applications. The core idea is to decompose the complex task of prompt injection detection into multiple focused sub-tasks, each handled by a specialized SLM, and take the majority vote as the final decision. The proposed system is positioned inline between untrusted inputs and the target Large Language Model. By performing detection before prompt assembly and LLM invocation, the system ensures that malicious or suspicious inputs are identified with minimal latency, thereby supporting real-time application requirements.

5.2. Methodology and Architecture

The central hypothesis of this project is that a set of Small Language Models, when fine-tuned for narrowly defined detection tasks and executed sequentially, can collectively achieve accurate, low-latency prompt injection detection comparable to larger models, while remaining practical for real-world deployment.

5.3. Process Flow and Components Overview

1. Input: The system receives a raw, unstructured user prompt, and it may contain mixed content such as text, code, malicious prompt injection attempts, etc.
2. Processing: The same raw input is processed sequentially step-by-step using multiple specialist Small Language Models (SLM A, SLM B, and SLM C). Each SLM operates independently and performs a specific detection task.
 - a. SLM A (gemma3-1b-it) - Analyse the given prompt and check detect semantic intent to override system behavior is there
 - b. SLM B (gemma3-1b-it) - Analyse the given prompt and check any privilege escalations like policy bypass attempts and requests for system prompts
 - c. SLM C (gemma3-1b-it) - Analyse the given prompt and check if any hidden attempts like encoding tricks, delimiters, prompt splitting, and indirect phrasing exist
3. Output: Majority Vote from above 3 results

6. Resource Requirements

6.1 Hardware Resources

- Primary Workstation: A laptop equipped with an Intel Core i7 or AMD Ryzen 7 processor, 16GB RAM, and NVIDIA GeForce RTX 306 GPU to handle local development and initial model testing.

- Cloud Computing: Access to Google Colab Pro will be utilized for fine-tuning SLMs to ensure computational efficiency during the intensive optimization phases.

6.2 Software and Data Resources

- Development Environment: Python 3.12+
- Libraries: PyTorch, HuggingFace transformers, peft.
- Datasets: Publicly available prompt injection datasets, natural language instructions datasets

7. Summary

This proposal presents a Small Language Model-based framework for detecting prompt injection attacks in LLM-integrated applications. The system emphasizes real-time and low computational overhead. By positioning the detector as a lightweight security guardrail, the proposed solution aims to enhance the safety and reliability of real-world LLM applications while remaining practical for widespread adoption.

8. References

- [1] P. Zhang, G. Zeng, T. Wang, and W. Lu, "TinyLlama: An Open-Source Small Language Model," Jun. 04, 2024, *arXiv*: arXiv:2401.02385. doi: 10.48550/arXiv.2401.02385.
- [2] Q. Zhang, Z. Liu, and S. Pan, "The Rise of Small Language Models," *IEEE Intell. Syst.*, vol. 40, no. 1, pp. 30–37, Jan. 2025, doi: 10.1109/MIS.2024.3517792.
- [3] T. H. Sakib, M. T. Hosain, and M. K. Morol, "Small Language Models: Architectures, Techniques, Evaluation, Problems and Future Adaptation," May 29, 2025, *arXiv*: arXiv:2505.19529. doi: 10.48550/arXiv.2505.19529.
- [4] R. Sharma and M. Mehta, "Small Language Models for Agentic Systems: A Survey of Architectures, Capabilities, and Deployment Trade offs," Oct. 04, 2025, *arXiv*: arXiv:2510.03847. doi: 10.48550/arXiv.2510.03847.
- [5] P. Belcak *et al.*, "Small Language Models are the Future of Agentic AI," Sep. 15, 2025, *arXiv*: arXiv:2506.02153. doi: 10.48550/arXiv.2506.02153.
- [6] S. Subramanian, V. Elango, and M. Gungor, "Small Language Models (SLMs) Can Still Pack a Punch: A survey," Jan. 03, 2025, *arXiv*: arXiv:2501.05465. doi: 10.48550/arXiv.2501.05465.
- [7] X. Suo, "Signed-Prompt: A New Approach to Prevent Prompt Injection Attacks Against LLM-Integrated Applications," Jan. 15, 2024, *arXiv*: arXiv:2401.07612. doi: 10.48550/arXiv.2401.07612.
- [8] D. Jacob, H. Alzahrani, Z. Hu, B. Alomair, and D. Wagner, "PromptShield: Deployable Detection for Prompt Injection Attacks," Apr. 12, 2025, *arXiv*: arXiv:2501.15145. doi: 10.48550/arXiv.2501.15145.
- [9] J. Pan, S. L. Wong, Y. Yuan, and X. W. Chia, "Prompt Inject Detection with Generative Explanation as an Investigative Tool," Feb. 16, 2025, *arXiv*: arXiv:2502.11006. doi: 10.48550/arXiv.2502.11006.
- [10] S. Hu *et al.*, "MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies," Jun. 03, 2024, *arXiv*: arXiv:2404.06395. doi: 10.48550/arXiv.2404.06395.

- [11] T. Schick and H. Schütze, “It’s Not Just Size That Matters: Small Language Models Are Also Few-Shot Learners,” Apr. 12, 2021, *arXiv*: arXiv:2009.07118. doi: 10.48550/arXiv.2009.07118.
- [12] S. Shaheer, G. M. R. Islam, M. R. Hamid, M. A. F. Khan, M. O. Faruk, and Y. Nur, “Detecting Prompt Injection Attacks Against Application Using Classifiers,” Dec. 14, 2025, *arXiv*: arXiv:2512.12583. doi: 10.48550/arXiv.2512.12583.
- [13] N. Maloyan, B. Ashinov, and D. Namiot, “Investigating the Vulnerability of LLM-as-a-Judge Architectures to Prompt-Injection Attacks,” May 19, 2025, *arXiv*: arXiv:2505.13348. doi: 10.48550/arXiv.2505.13348.
- [14] R. Zhang, D. Sullivan, K. Jackson, P. Xie, and M. Chen, “Defense against Prompt Injection Attacks via Mixture of Encodings,” Apr. 10, 2025, *arXiv*: arXiv:2504.07467. doi: 10.48550/arXiv.2504.07467.
- [15] K.-H. Hung, C.-Y. Ko, A. Rawat, I.-H. Chung, W. H. Hsu, and P.-Y. Chen, “Attention Tracker: Detecting Prompt Injection Attacks in LLMs,” Apr. 23, 2025, *arXiv*: arXiv:2411.00348. doi: 10.48550/arXiv.2411.00348.
- [16] Y. Chen, H. Li, Z. Zheng, Y. Song, D. Wu, and B. Hooi, “Defense Against Prompt Injection Attack by Leveraging Attack Techniques,” Aug. 02, 2025, *arXiv*: arXiv:2411.00459. doi: 10.48550/arXiv.2411.00459.
- [17] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong, “DataSentinel: A Game-Theoretic Detection of Prompt Injection Attacks,” Nov. 12, 2025, *arXiv*: arXiv:2504.11358. doi: 10.48550/arXiv.2504.11358.

9. Appendix

Activity	Jan				Feb				March				April				May				June				July			
	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4
Literature Review & Proposal Submission																												
Data Collection & Pre-processing																												
SLM Selection & Model Fine-tuning																												
Evaluation & Performance Analysis																												
Final Thesis & Project Submission																												

Figure 2. Gaant Chart